

---

# Rapport conceptionnel

## Mini-Projet Système d'Exploitation

---

### Environnement de Développement Web avec Docker sous Linux

Réalisé par :

EL HORRE OMAR

EL GHAZOUANI MAROUANE

CHERRADI ILYASS

BOUARGUAN ABDELLAH

NIZAR BEN YACOUB

Filière : Génie Informatique 1

Encadrant : JABER EL BOUHDIDI

Année : 2025 / 2026

Date : Mai 2026

---

# Table des matières

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                | <b>3</b>  |
| 1.1      | Contexte et justification du projet . . . . .                      | 3         |
| 1.2      | Problématique de départ (environnements hétérogènes) . . . . .     | 3         |
| 1.3      | Diversité des systèmes d'exploitation . . . . .                    | 4         |
| 1.4      | Objectifs du projet . . . . .                                      | 4         |
| 1.4.1    | Objectifs principaux . . . . .                                     | 4         |
| 1.4.2    | Objectifs pédagogiques . . . . .                                   | 4         |
| 1.5      | Divergences dans les configurations des services annexes . . . . . | 5         |
| 1.6      | Aperçu des outils et technologies utilisés . . . . .               | 5         |
| 1.6.1    | Docker et conteneurisation . . . . .                               | 5         |
| 1.6.2    | Technologies web . . . . .                                         | 5         |
| 1.6.3    | Collaboration et gestion . . . . .                                 | 5         |
| <b>2</b> | <b>Organisation de l'équipe et gestion du projet</b>               | <b>7</b>  |
| 2.1      | Présentation de l'équipe . . . . .                                 | 7         |
| 2.2      | Méthodologie adoptée . . . . .                                     | 8         |
| 2.2.1    | Avantages de l'approche collaborative . . . . .                    | 8         |
| 2.3      | Planification des tâches et livrables . . . . .                    | 8         |
| 2.3.1    | Diagramme de Gantt . . . . .                                       | 9         |
| 2.4      | Outils de communication . . . . .                                  | 10        |
| <b>3</b> | <b>Conception de l'application</b>                                 | <b>11</b> |
| 3.1      | Diagramme des cas d'utilisation . . . . .                          | 11        |
| 3.1.1    | Identification des acteurs . . . . .                               | 11        |
| 3.1.2    | Représentation graphique . . . . .                                 | 11        |
| <b>4</b> | <b>Choix technologiques</b>                                        | <b>13</b> |
| 4.1      | Pourquoi Docker ? . . . . .                                        | 13        |
| 4.2      | Pourquoi GitHub ? . . . . .                                        | 14        |
| 4.3      | Pourquoi Jira ? . . . . .                                          | 14        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 4.4      | Pourquoi Terraform ? . . . . .                                 | 15        |
| 4.5      | Stack technique retenue . . . . .                              | 16        |
| 4.5.1    | PHP (Backend) . . . . .                                        | 16        |
| 4.5.2    | HTML / CSS / JavaScript (Frontend) . . . . .                   | 16        |
| 4.5.3    | PostgreSQL (Base de données) . . . . .                         | 17        |
| 4.5.4    | Nginx (Serveur web) . . . . .                                  | 17        |
| 4.5.5    | Avantages de cette stack dans l'environnement Docker . . . . . | 17        |
| <b>5</b> | <b>Intégration et Déploiement Continu (CI/CD)</b>              | <b>18</b> |
| 5.1      | Mise en place de GitHub Actions . . . . .                      | 18        |
| 5.1.1    | Configuration du Workflow . . . . .                            | 18        |
| 5.1.2    | Déclencheurs (Triggers) . . . . .                              | 18        |
| <b>6</b> | <b>Développement de l'application web</b>                      | <b>19</b> |
| 6.1      | Sujet choisi (description de l'application) . . . . .          | 19        |
| 6.2      | Fonctionnalités principales . . . . .                          | 19        |
| 6.3      | Interface utilisateur . . . . .                                | 20        |
| 6.3.1    | Page de connexion . . . . .                                    | 20        |
| 6.3.2    | Tableau de bord . . . . .                                      | 21        |
| 6.3.3    | Page des leçons . . . . .                                      | 22        |
| 6.3.4    | Interface administrateur . . . . .                             | 23        |
| 6.3.5    | Lecture de leçon . . . . .                                     | 23        |
| <b>7</b> | <b>Conclusion Générale</b>                                     | <b>25</b> |
| 7.1      | Résumé du projet . . . . .                                     | 25        |
| 7.2      | Objectifs atteints . . . . .                                   | 25        |
| 7.3      | Perspectives d'évolution . . . . .                             | 26        |

# Chapitre 1

## Introduction

### 1.1 Contexte et justification du projet

Dans le cadre du module Système d'exploitation de la première année du cycle Génie Informatique (2025-2026), notre équipe a réalisé un mini-projet visant à créer un environnement de développement web collaboratif basé sur Docker sous Linux. Ce projet, qui s'inscrit dans une démarche pédagogique axée sur les technologies DevOps modernes, nous a permis de développer une application web d'apprentissage nommée « INFOENSA », avec PHP/Laravel pour le backend et HTML/CSS/JavaScript pour le frontend. L'objectif principal était de nous familiariser avec les outils et pratiques utilisés en entreprise pour le travail en équipe, notamment la standardisation des environnements de développement via Docker, le versionnement du code avec GitHub et la gestion des tâches avec Jira. Face aux défis techniques et organisationnels que pose le développement collaboratif, notamment l'harmonisation des environnements de travail, la conteneurisation offre une solution efficace pour assurer la cohérence entre les phases de développement, de test et de production. Ce rapport détaille notre démarche, les difficultés rencontrées et les solutions mises en œuvre.

### 1.2 Problématique de départ (environnements hétérogènes)

La problématique fondamentale adressée par ce projet découle d'une réalité opérationnelle fréquemment observée dans les équipes de développement web : l'hétérogénéité des environnements de travail constitue un obstacle majeur à l'efficacité et à la cohérence du processus de développement collaboratif. Cette hétérogénéité se manifeste à plusieurs niveaux : systèmes d'exploitation différents (Linux, Windows, Mac), disparités dans les

versions d'API et de frameworks (PHP7/Laravel 7 vs PHP8/Laravel 10, etc.), ainsi que des difficultés d'intégration lors de la fusion des tâches. Pour résoudre ces problèmes, nous avons mis en place une solution basée sur Docker garantissant que tous les membres de l'équipe travaillent dans un environnement standardisé, indépendamment de leur système d'exploitation.

## 1.3 Diversité des systèmes d'exploitation

Les équipes de développement utilisent généralement différents systèmes d'exploitation :

- **Linux** : Diverses distributions (Ubuntu, Debian, ManjaroLinux) avec des configurations système spécifiques.
- **Windows** : Différentes versions (10, 11) avec des comportements variables concernant les chemins de fichiers, les permissions et les services.
- **macOS** : Versions multiples avec des particularités propres à l'écosystème Apple.

## 1.4 Objectifs du projet

Face à la problématique des environnements hétérogènes, notre projet poursuit plusieurs objectifs :

### 1.4.1 Objectifs principaux

1. **Standardisation** : Créer un environnement unifié (PHP/Laravel et Angular), éliminer le syndrome du « Works on my machine ».
2. **Infrastructure DevOps** : Implémenter GitHub pour le versionnement et Jira pour la gestion de projet.
3. **Développement et déploiement** : Développer une application d'apprentissage (« INFOENSA ») et assurer la reproductibilité entre développement et production.

### 1.4.2 Objectifs pédagogiques

1. **Compétences techniques** : Maîtriser Docker sous Linux, principes CI/CD et méthodologies agiles.
2. **Workflow professionnel** : Simuler un environnement d'entreprise, collaboration via GitHub/Jira et communication technique.

## 1.5 Divergences dans les configurations des services annexes

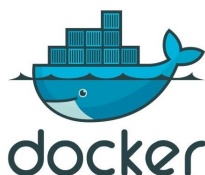
Nous avons observé des disparités majeures :

- **Serveurs web** : Apache vs Nginx.
- **Bases de données** : Versions de PostgreSQL avec des dialectes SQL spécifiques.

Ces divergences entraînent des problèmes d'intégration, des conflits de dépendances, des ruptures de compatibilité et des surcoûts de maintenance.

## 1.6 Aperçu des outils et technologies utilisés

### 1.6.1 Docker et conteneurisation



- **Docker** : Plateforme de conteneurisation.
- **Docker Compose** : Gestion d'applications multi-conteneurs.
- **Docker Hub** : Registre d'images.

### 1.6.2 Technologies web

Backend (PHP), Frontend (HTML, CSS, JS), Base de données (PostgreSQL), Serveur web (Nginx).

### 1.6.3 Collaboration et gestion



- **GitHub** : Gestion de versions (Branches, Pull Requests, Actions).



— **Jira** : Suivi de projet (Backlog, Sprints, Kanban).

# Chapitre 2

## Organisation de l'équipe et gestion du projet

### 2.1 Présentation de l'équipe

- **OMAR EL HORRE - Chef de projet**

*Responsabilités* : Configuration de Jira, organisation du projet, répartition des tâches, suivi de l'avancement, validation après code review et test, organisation des réunions, envoi des projets, intervention en cas de problème, participation au développement.

- **Marouane EL GHAZOUANI - Responsable du développement**

*Responsabilités* : Développement frontend, développement backend.

- **Abdellah BOUARGUAN - Responsable GitHub, Tests et CI/CD**

*Responsabilités* : Configuration GitHub et gestion des versions (Git), gestion des branches et Pull Requests, mise en place des tests (unitaires et d'intégration), participation au déploiement automatique, participation au développement (Backend).

- **Nizar BEN YACOUB - Responsable Docker, BDD et Déploiement**

*Responsabilités* : Création des images Docker, configuration des services (Frontend, PostgreSQL), gestion de la base de données, participation au déploiement (avec Terraform).

- **Ilyass CHERRADI - Responsable Déploiement (Terraform) et Support**

*Responsabilités* : Configuration de Terraform, création du serveur Linux, déploiement final de l'application, support Docker, participation au développement.



## 2.2 Méthodologie adoptée

Pour réaliser ce projet, nous avons mis en place une organisation simple basée sur l'entraide, inspirée des méthodes Agiles. Nous avons utilisé Jira pour lister et suivre nos tâches. Plutôt que de séparer strictement les rôles, nous avons favorisé une approche collective :

1. **Travail en groupe** : Réunions régulières pour avancer ensemble sur les fonctionnalités.
2. **Binômes sur le code** : Développement en binôme pour réduire les erreurs et favoriser l'apprentissage mutuel.
3. **Polyvalence** : Chaque membre a contribué au backend, au frontend et à Docker pour une vision globale.
4. **Résolution collective** : Traitement commun des difficultés techniques pour accélérer la résolution des problèmes.

### 2.2.1 Avantages de l'approche collaborative

Cette approche a présenté plusieurs bénéfices :

- **Meilleure compréhension** : Vision holistique du fonctionnement du projet.
- **Réduction des blocages** : Mise en commun des compétences pour une résolution rapide.
- **Partage des connaissances** : Apprentissage continu entre les membres.
- **Cohérence accrue** : Implication transversale garantissant une meilleure unité de l'application.

## 2.3 Planification des tâches et livrables

Le projet a été structuré via l'outil de gestion Jira, permettant un suivi précis des étapes. La figure ci-dessous illustre notre gestion de projet :

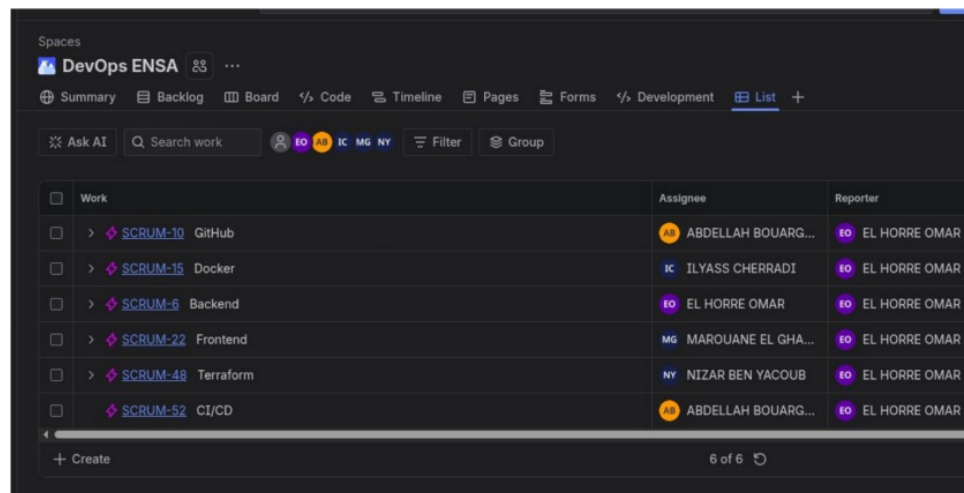


FIGURE 2.1 – Capture d’écran de la gestion de projet sur Jira

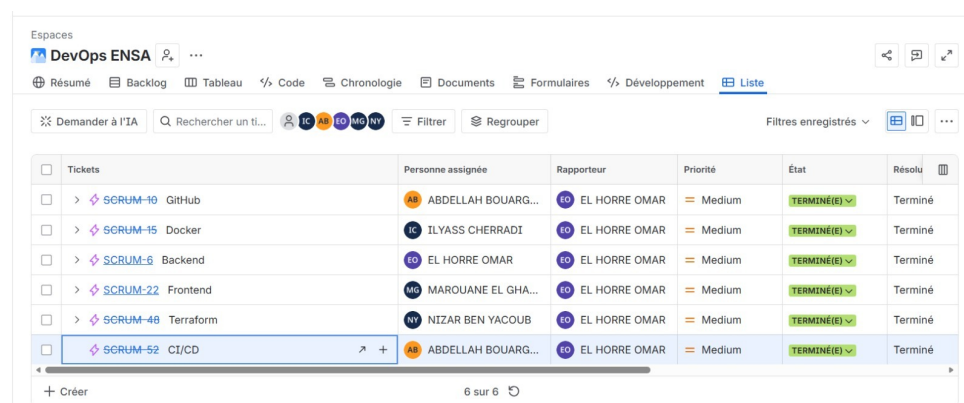


FIGURE 2.2 – Capture d’écran de la gestion de projet sur Jira

### 2.3.1 Diagramme de Gantt

Afin de visualiser la répartition temporelle de nos activités sur la période du 01/04/2026 au 21/05/2026, nous avons établi un diagramme de Gantt. Celui-ci détaille les phases clés du projet, allant de l’autoformation et la configuration des environnements (Docker, GitHub) jusqu’au déploiement final et la rédaction des livrables.

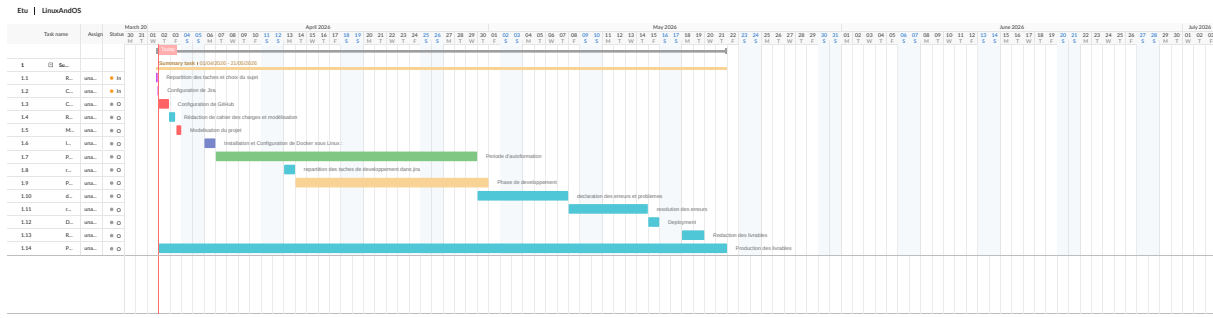


FIGURE 2.3 – Diagramme de Gantt du projet « INFOENSA »

## 2.4 Outils de communication

Pour assurer une communication efficace, nous avons utilisé les outils suivants :

- **Jira** : Gestion des tâches et suivi de l'avancement.
- **GitHub** : Collaboration sur le code source et versionnement.
- **Google Meet** : Réunions de synchronisation d'équipe.
- **WhatsApp** : Communication rapide et échanges quotidiens.

# Chapitre 3

## Conception de l'application

### 3.1 Diagramme des cas d'utilisation

Afin de bien cerner les fonctionnalités offertes par notre plateforme d'apprentissage « INFOENSA » et d'identifier les interactions entre les différents utilisateurs et le système, nous avons modélisé un diagramme de cas d'utilisation.

#### 3.1.1 Identification des acteurs

Nous avons identifié trois acteurs principaux interagissant avec notre système :

- **L'Administrateur** : Il est responsable de la gestion du contenu pédagogique et de l'administration générale de la plateforme. Il possède les privilèges nécessaires pour ajouter ou supprimer des leçons.
- **L'Étudiant** : C'est l'utilisateur cible de la plateforme. Il interagit avec le système pour s'inscrire, s'authentifier, gérer son apprentissage (sélectionner, lire, terminer ou reprendre une leçon), participer au forum, consulter son profil, suivre ses classements et passer des quiz pour valider ses acquis.
- **Le système GEMINI API** : Il s'agit d'un acteur secondaire (système externe) sollicité de manière automatisée lors du déroulement des quiz (notamment pour l'évaluation, la génération ou le traitement de données liées à l'intelligence artificielle).

#### 3.1.2 Représentation graphique

La figure ci-dessous illustre l'ensemble des interactions prévues dans le système. Elle met en évidence les différentes dépendances entre les cas d'utilisation, notamment les relations d'inclusion (*include*) telles que la sélection préalable d'un semestre et d'un mo-

dule avant de sélectionner une leçon, ainsi que les relations d'extension (*extend*) pour les fonctionnalités optionnelles ou dérivées.

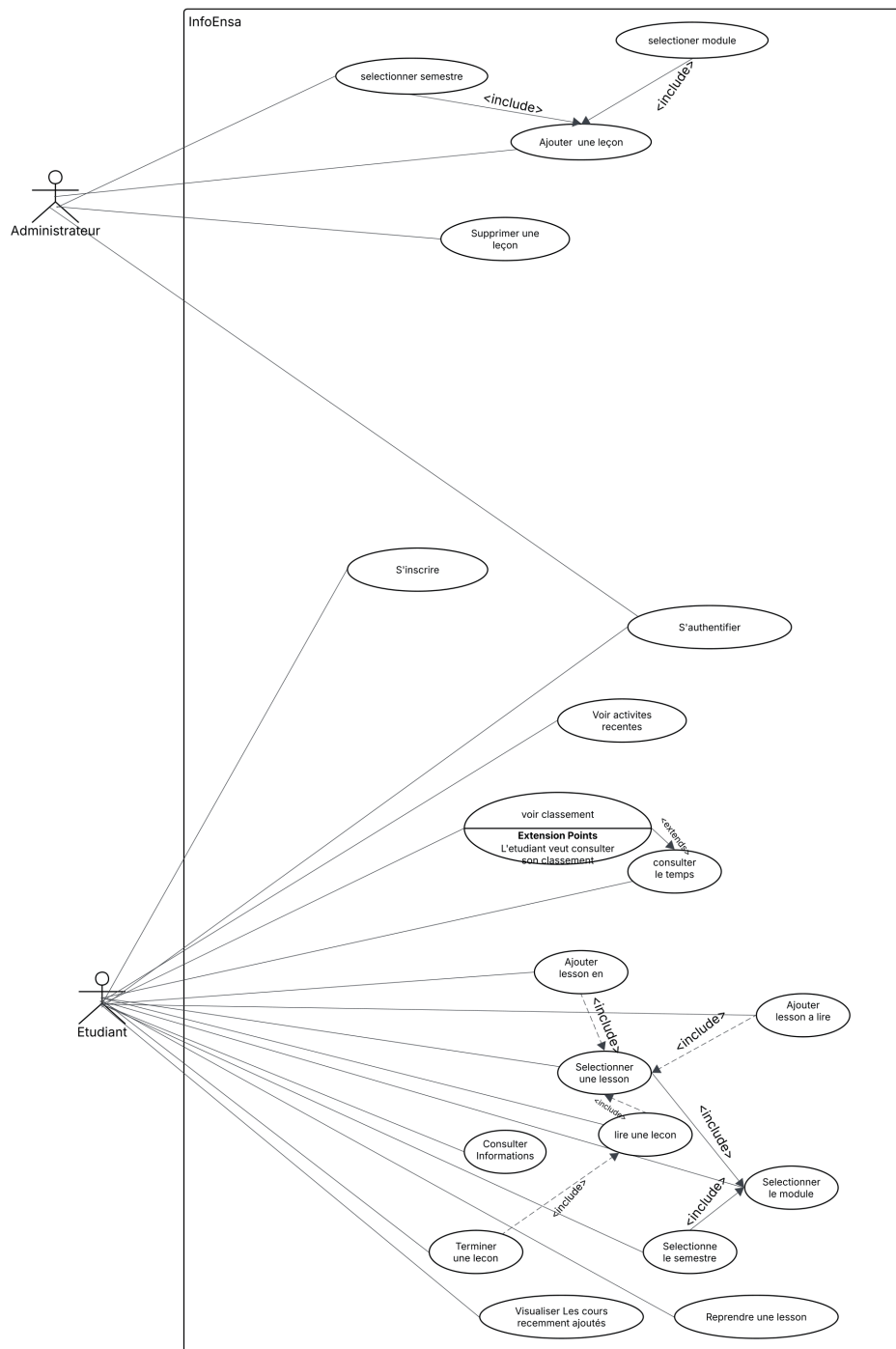
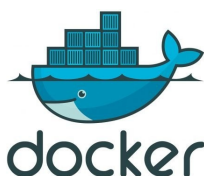


FIGURE 3.1 – Diagramme de cas d'utilisation du système INFOENSA

# Chapitre 4

## Choix technologiques

### 4.1 Pourquoi Docker ?



Nous avons choisi Docker pour les raisons suivantes :

- **Même environnement pour toute l'équipe** : Docker permet de créer des conteneurs (sortes de boîtes isolées) qui contiennent tout ce qu'il faut pour faire tourner l'application (PHP, Laravel, Angular, base de données). Ainsi, chaque membre de l'équipe travaille dans le même environnement, même si sa machine est sous Windows, Mac ou Linux.
- **Services bien séparés** : Chaque service (backend, frontend, base de données) vit dans son propre conteneur. Cela rend plus facile la recherche d'erreurs et les mises à jour.
- **Déploiement simplifié** : Ce qui fonctionne en développement fonctionne aussi en production, car l'environnement est le même. Fini le fameux problème « ça marche sur ma machine, mais pas sur le serveur ».
- **Gain de temps** : Un nouveau membre dans l'équipe peut commencer à travailler tout de suite, sans passer des heures à configurer son ordinateur.
- **Facile à agrandir** : Si l'application a beaucoup d'utilisateurs, on peut facilement créer plusieurs conteneurs pour gérer la charge.

## 4.2 Pourquoi GitHub ?



GitHub est la plateforme idéale pour le versionnement de code et la collaboration pour notre projet pour les raisons suivantes :

- **Contrôle de version Git** : GitHub utilise Git, le système de contrôle de version le plus puissant et le plus adopté, permettant de suivre précisément l'évolution du code et de gérer efficacement les branches.
- **Collaboration structurée** : Les Pull Requests permettent une révision systématique du code avant intégration, améliorant la qualité globale et réduisant les bugs.
- **Gestion des branches** : GitHub facilite la gestion des branches pour travailler simultanément sur différentes fonctionnalités sans interférence entre les développeurs.
- **Intégration avec CI/CD** : GitHub Actions permet d'automatiser les tests et le déploiement directement depuis les dépôts, assurant la qualité du code à chaque commit.
- **Documentation intégrée** : GitHub permet de maintenir une documentation à jour via les fichiers README.md.
- **Suivi des problèmes** : Le système d'issues de GitHub permet de suivre les bugs et les demandes de fonctionnalités de manière centralisée.
- **Intégration avec Jira** : GitHub s'intègre facilement avec ces outils de gestion de projet, créant un écosystème complet pour la gestion du développement.

## 4.3 Pourquoi Jira ?



Ces outils de gestion de projet sont essentiels pour une collaboration efficace pour les raisons suivantes :

- **Gestion agile du projet** : Jira est spécialement conçu pour la méthodologie Agile, avec des tableaux Kanban et des sprints Scrum qui structurent le travail en cycles de développement.
- **Suivi précis des tâches** : Ces outils permettent de décomposer le projet en tâches spécifiques, d'assigner des responsables et de suivre leur progression en temps réel.
- **Priorisation claire** : La hiérarchisation des tâches permet à l'équipe de se concentrer sur les fonctionnalités les plus importantes d'abord.
- **Communication centralisée** : Les commentaires et discussions sur les tâches sont conservés au même endroit, créant un historique consultable.
- **Rapports et métriques** : Jira fournit des graphiques (burndown charts, velocity charts) qui permettent d'analyser la productivité de l'équipe et d'ajuster les estimations.
- **Intégration avec GitHub** : Les commits peuvent être liés automatiquement aux tâches Jira, créant une traçabilité complète entre les exigences et l'implémentation.
- **Documentation des décisions** : Notion excelle particulièrement dans la création d'une base de connaissances pour documenter les décisions techniques et les processus.
- **Visibilité pour les parties prenantes** : Les tableaux de bord personnalisables permettent aux gestionnaires de projet et aux clients de suivre l'avancement sans interrompre les développeurs.

## 4.4 Pourquoi Terraform ?



Bien que Docker ait été utilisé pour conteneuriser notre application, nous avons également choisi d'introduire Terraform pour une raison précise : automatiser et standardiser l'infrastructure qui accueille nos conteneurs. Voici pourquoi :

- **Infrastructure comme code** : Terraform permet de décrire toute l'infrastructure (serveurs, réseaux, bases de données, etc.) dans des fichiers de configuration. On peut ainsi la stocker dans GitHub, la relire, la modifier et la partager facilement.



- **Environnements identiques** : Avec Terraform, on peut créer le même environnement sur différents serveurs (développement, test, production) sans rien faire à la main. Cela évite les erreurs dues à des configurations manuelles différentes.
- **Déploiement reproductible** : Si un serveur plante ou doit être recréé, il suffit d'exécuter une commande Terraform pour reconstruire toute l'infrastructure à l'identique. Plus besoin de se souvenir de ce qui a été installé.
- **Gestion simplifiée des ressources** : Terraform gère tout seul l'ordre de création des ressources (par exemple, d'abord le réseau, puis le serveur, puis Docker). Si une ressource échoue, il ne continue pas, ce qui évite les erreurs en cascade.
- **Travail d'équipe facilité** : Comme les fichiers Terraform sont versionnés sur GitHub, toute l'équipe peut voir, modifier et améliorer l'infrastructure. On peut également faire des revues de code (Pull Requests) sur les changements d'infrastructure, comme pour le code de l'application.
- **Multi-plateforme** : Terraform fonctionne avec de nombreux fournisseurs (cloud, serveurs sur site, conteneurs Docker). Dans notre projet, nous l'avons utilisé pour déployer et gérer nos conteneurs Docker sur un serveur Linux de manière automatisée.
- **Évolution propre** : Quand on modifie l'infrastructure (ajouter un conteneur, changer une version, augmenter la mémoire), Terraform calcule uniquement ce qui a changé et applique les modifications sans casser ce qui fonctionne déjà.

## 4.5 Stack technique retenue

Notre stack technique a été sélectionnée pour sa robustesse, sa flexibilité et sa compatibilité avec Docker :

### 4.5.1 PHP (Backend)

- Langage de programmation serveur éprouvé et largement utilisé.
- Excellente intégration avec PostgreSQL et Nginx.
- Performance optimale pour les applications web dynamiques.
- Grande communauté et documentation abondante.

### 4.5.2 HTML / CSS / JavaScript (Frontend)

- **HTML5** pour la structure sémantique des pages.

- **CSS3** pour la mise en forme responsive et les animations.
- **JavaScript** pour l'interactivité côté client.
- Compatibilité universelle avec tous les navigateurs modernes.

#### 4.5.3 PostgreSQL (Base de données)

- Système de gestion de base de données relationnelle fiable.
- Performances optimisées pour les applications web.
- Excellente intégration avec PHP.
- Facilement conteneurisable avec Docker.

#### 4.5.4 Nginx (Serveur web)

- Performances supérieures avec une faible empreinte mémoire.
- Configuration optimisée pour PHP via PHP-FPM.
- Excellente gestion des connexions concurrentes.

#### 4.5.5 Avantages de cette stack dans l'environnement Docker

- Architecture simple et efficace pour applications web traditionnelles.
- Tous les composants peuvent être facilement conteneurisés.
- Permet un développement frontal et back-end dans un environnement unifié.
- Élimine les problèmes de compatibilité entre développeurs.
- Configuration identique en développement et en production.

Cette stack technique offre un excellent équilibre entre simplicité d'utilisation, performances et flexibilité, tout en étant parfaitement adaptée à la conteneurisation avec Docker.

# Chapitre 5

## Intégration et Déploiement Continu (CI/CD)

### 5.1 Mise en place de GitHub Actions

Dans notre projet, nous avons mis en place une Intégration Continue (CI) grâce à **GitHub Actions**, afin d'automatiser le processus de test et de build de notre application conjointe (Laravel pour le backend et Angular pour le frontend).

Cette automatisation nous permet de garantir que chaque modification apportée au code source est testée et validée avant d'être déployée, réduisant ainsi considérablement les risques de régression.

#### 5.1.1 Configuration du Workflow

Le fichier de configuration définissant les étapes de notre pipeline CI/CD est situé dans notre dépôt à l'emplacement suivant :

```
.github/workflows/deploy.yml
```

#### 5.1.2 Déclencheurs (Triggers)

Pour assurer une fluidité dans le développement et éviter les lancements inutiles, le workflow est paramétré pour être exécuté automatiquement lors d'événements spécifiques :

- Lors d'un **push** (envoi de code) direct sur la branche principale (**main**).

# Chapitre 6

## Développement de l'application web

### 6.1 Sujet choisi (description de l'application)

Le sujet choisi consiste à développer une plateforme web d'apprentissage destinée aux étudiants de Génie Informatique. Cette application permet aux utilisateurs de consulter facilement les cours et les leçons selon l'année, le semestre et le module sélectionné. La plateforme offre également plusieurs fonctionnalités interactives telles que la gestion des favoris, la reprise de lecture, le suivi du temps passé sur chaque leçon ainsi que la consultation des activités récentes.

Afin d'améliorer l'expérience d'apprentissage, l'application intègre l'API Gemini pour générer automatiquement des résumés de cours et des quiz interactifs. Un espace administrateur est également prévu pour permettre la gestion des contenus pédagogiques et l'ajout des nouvelles leçons.

### 6.2 Fonctionnalités principales

L'application offre plusieurs fonctionnalités principales pour les utilisateurs :

- S'inscrire et se connecter à la plateforme (authentification).
- Accéder à un tableau de bord personnalisé.
- Consulter les cours récemment ajoutés.
- Reprendre la lecture d'une leçon à partir du dernier point consulté.
- Parcourir les leçons par année, semestre et module.
- Lire des leçons au format PDF.
- Ajouter des leçons aux favoris.
- Ajouter des leçons à la liste « À lire plus tard ».

- Générer automatiquement des résumés de cours grâce à l'API Gemini.
- Générer automatiquement des quiz après lecture d'une leçon.
- Enregistrer les scores obtenus aux quiz.
- Consulter le temps passé sur chaque leçon.
- Accéder à un système de classement basé sur le temps de lecture quotidien.
- Consulter son profil personnel et l'historique des activités récentes.

## 6.3 Interface utilisateur

### 6.3.1 Page de connexion

Cette interface permet à l'étudiant de se connecter à l'aide de son adresse e-mail institutionnelle et d'un mot de passe, ou de créer un nouveau compte afin d'accéder aux différentes fonctionnalités de la plateforme.

ENSATE / GI  
Plateforme de préparation, Génie Informatique.

### Connexion

Accédez à votre espace étudiant

Adresse email

prenom.nom@etu.uae.ac.ma  
Please fill out this field.

Mot de passe

\*\*\*\*\*

**Se connecter**

Pas encore de compte ? [Créer un compte](#)

FIGURE 6.1 – Interface de connexion à la plateforme INFOENSA

15.237.184.157:4200/auth.html

ENSATE / GI  
Plateforme de préparation, Génie Informatique.

## Inscription

Créez votre compte étudiant ENSATE

Nom complet

Adresse email

Mot de passe

**S'inscrire**

Déjà inscrit ? [Se connecter](#)

FIGURE 6.2 – Interface de creation de compte à la plateforme INFOENSA

### 6.3.2 Tableau de bord

Cette interface accueille l'étudiant après son authentification et lui permet d'accéder rapidement à la bibliothèque de leçons, aux ressources pédagogiques ainsi qu'aux cours récemment ajoutés.

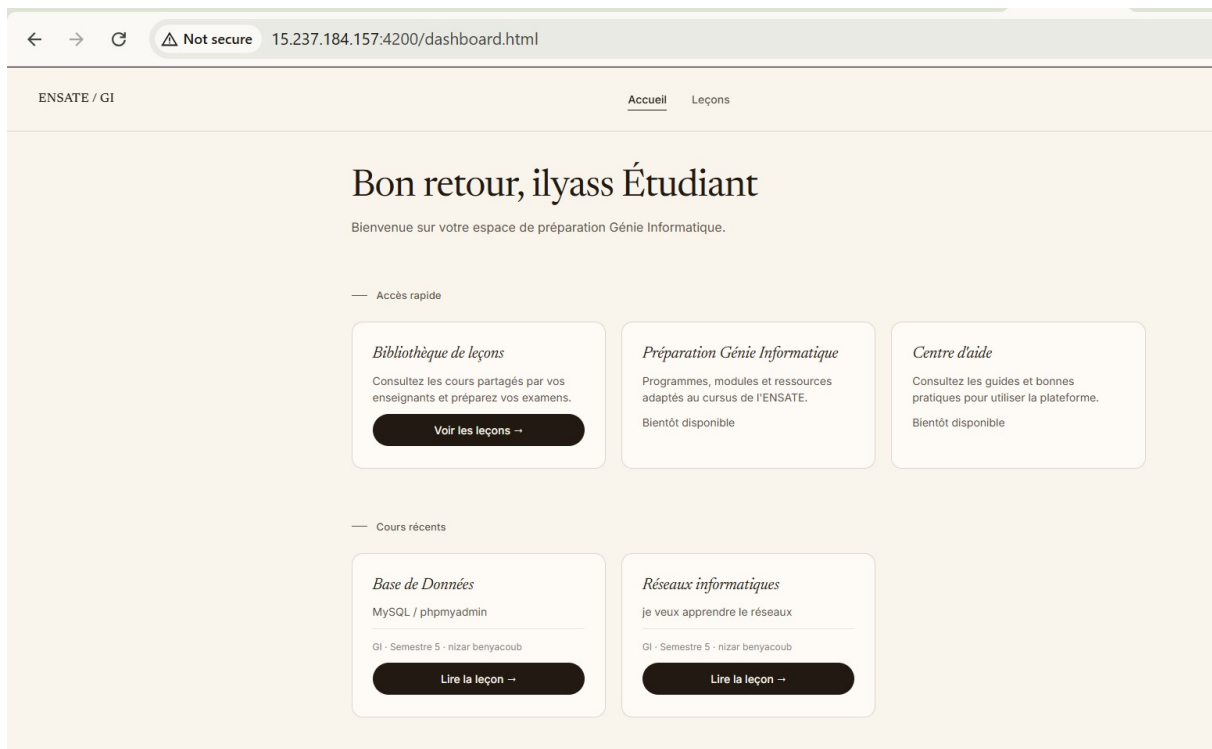


FIGURE 6.3 – Tableau de bord de l'utilisateur

### 6.3.3 Page des leçons

Cette interface permet de sélectionner un semestre et un cours afin d'accéder aux différentes leçons disponibles.

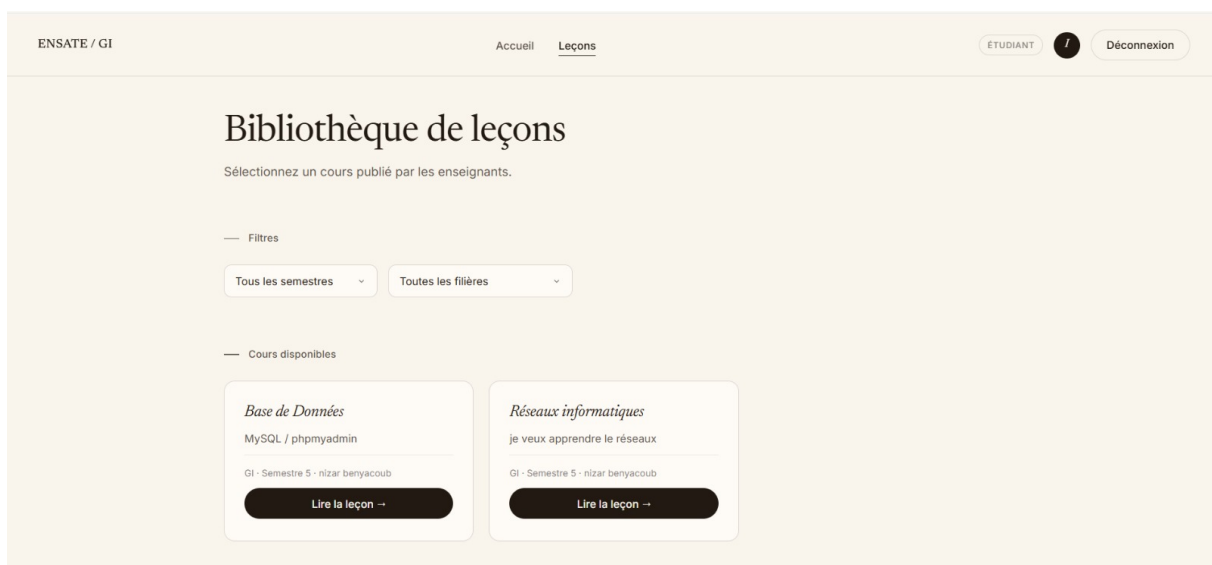


FIGURE 6.4 – Interface de sélection des leçons par semestre et par cours

### 6.3.4 Interface administrateur

L'administrateur peut ajouter, modifier et gérer les leçons ainsi que les ressources pédagogiques de la plateforme de manière centralisée.

ENSATE / GI    Accueil    Leçons    ADMINISTRATEUR    N    Déconnexion

## Bibliothèque de leçons

Sélectionnez un cours publié par les enseignants.

Ajouter une nouvelle leçon

Titre de la leçon    Filière    Semestre

Description (optionnelle)    Lien Google Drive (optionnel)

Glissez un fichier ici  
ou cliquez pour parcourir vos fichiers

Publier la leçon

FIGURE 6.5 – Interface d'administration pour la gestion des contenus pédagogiques

### 6.3.5 Lecture de leçon

L'étudiant peut consulter une leçon au format PDF avec un suivi automatique du temps de lecture.



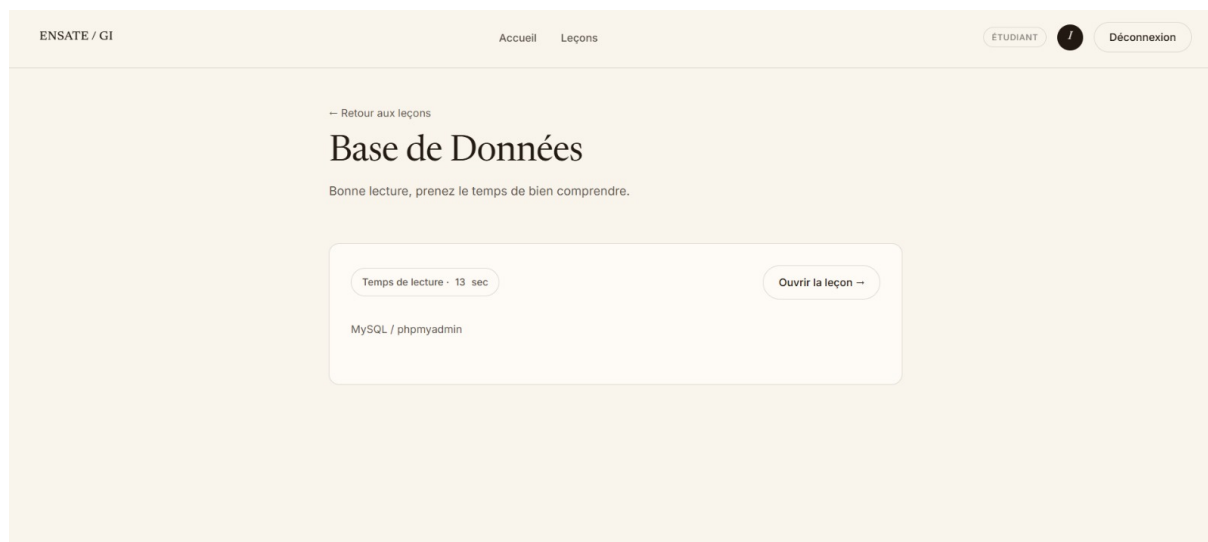


FIGURE 6.6 – Interface de lecture d’une leçon au format PDF avec suivi du temps et il peut aussi consulter le cours dans google drive

# Chapitre 7

## Conclusion Générale

### 7.1 Résumé du projet

Ce mini-projet de Système d'Exploitation nous a permis de concevoir et de développer « INFOENSA », une plateforme web d'apprentissage interactive dédiée aux étudiants en Génie Informatique. Au-delà du développement applicatif basé sur une stack robuste (PHP, HTML/CSS/JavaScript, PostgreSQL et Nginx), l'accent a été mis sur la mise en œuvre d'une infrastructure DevOps moderne.

Grâce à la conteneurisation avec Docker sous Linux, à l'automatisation de l'infrastructure avec Terraform, au versionnement rigoureux sur GitHub et au pipeline CI/CD via GitHub Actions, nous avons pu simuler un environnement de production réel et surmonter la problématique des environnements hétérogènes au sein de l'équipe.

### 7.2 Objectifs atteints

L'évaluation du projet met en évidence la réalisation des objectifs tant techniques que pédagogiques :

- **Standardisation réussie** : L'utilisation de Docker a totalement éliminé les conflits de configuration entre les machines des développeurs (Windows, Mac, Linux).
- **Infrastructure as Code (IaC) opérationnelle** : L'intégration de Terraform a permis de rendre l'infrastructure hautement reproductible, prévisible et facilement gérable en équipe.
- **Automatisation et qualité** : Le workflow GitHub Actions configuré garantit une intégration continue fluide, testant automatiquement le code à chaque push sur la branche `main`.
- **Gestion de projet agile** : L'utilisation de Jira combinée à GitHub a structuré

notre collaboration, permettant un suivi précis des tâches et le respect du planning établi.

- **Application fonctionnelle** : Toutes les interfaces clés (Connexion, Tableau de bord, Page des leçons, Lecture de PDF avec suivi du temps, et Espace Administrateur) ont été implémentées avec succès.

## 7.3 Perspectives d'évolution

Bien que la plateforme soit pleinement fonctionnelle dans sa version actuelle, plusieurs axes d'amélioration sont envisagés pour la suite du projet :

- **Intégration de l'Intelligence Artificielle** : Finaliser l'intégration de l'API Gemini, qui a été modélisée lors de la conception, afin de pouvoir générer automatiquement des résumés de cours et des quiz interactifs.
- **Évolution de la Stack Frontend** : Poursuivre la transition de l'interface utilisateur vers un framework moderne comme Angular pour offrir une expérience Single Page Application (SPA) plus dynamique.
- **Diversification des formats** : Étendre le support pédagogique au-delà du format PDF en intégrant un système de streaming vidéo pour les cours magistraux enregistrés.
- **Déploiement à grande échelle** : Migrer l'infrastructure provisionnée par Terraform vers un fournisseur Cloud d'envergure (comme AWS, GCP ou Azure) afin de supporter une charge d'utilisateurs plus importante.